

Introduction & Setup

Week 1 · Foundations module · Textbook Chapter 1

Brian C. Keegan, Ph.D.

Associate Professor, Department of Information Science
University of Colorado Boulder

Friday, August 21, 2026

This week at a glance

Welcome to Web Data Science. The semester starts Thursday, so this week we meet **Friday only** — today is orientation, setup, and the plan for the semester.

Friday | Welcome, setup & how the class works

- What web data science is — and why it matters
- Get Python, Git, and a GitHub account working
- How the weekly rhythm and grading work

The **Monday / Wednesday / Friday** rhythm begins next week — see “The weekly rhythm.”

Companion notebook

`ch-01-introduction.ipynb`

Before Monday

Install Anaconda and create a **free** GitHub account. Bring a laptop — we work hands-on.

1. Welcome

Brian C. Keegan — Associate Professor of Information Science.

- Grew up outside Las Vegas, Nevada
- MIT: Mechanical Engineering and Science, Technology & Society
- A year as a bartender and oral historian
- Ph.D. at Northwestern (Media, Technology & Society); postdoc in computational social science at Northeastern
- Data scientist at Harvard Business School
- CU Boulder Information Science, 2016–present

brianckeegan.com

What I study

- How Wikipedia covers breaking news
- Public-interest data science
- Migration across platforms (Threads, Mastodon, Bluesky)
- Online extremism and moderation

My students and I rely on web data every day — but researchers' access to it is rapidly disappearing.

What is web data science?

Web data science is the practice of *retrieving* data from the web, *parsing* it into structured formats, and *analyzing* it to produce knowledge.

It differs from general data science in one crucial way: your first challenge is not modeling or visualization — it is **getting the data in the first place**.

The web is the largest, most dynamic, and most contested data source ever created. It can tell us how misinformation spreads, how housing costs shift, and how online communities govern themselves.

Not your usual dataset

The web is **not a database**. Data arrives as:

- HTML tables buried in tracking scripts
- JSON from rate-limited APIs
- scanned PDFs of council minutes
- posts that might vanish tomorrow

Web data science grew out of **computational social science**, **data journalism**, and **civic technology** — traditions that share one idea: data fluency is a form of power that should serve the public.

- **ProPublica, “Machine Bias” (2016)** — scraped court records exposed racial bias in criminal risk-assessment algorithms.
- **The Markup, “Citizen Browser”** — a recruited panel revealed how Facebook’s feed distributed political content.

The tools are not neutral

The same skills that make public information **accessible** can also enable surveillance. Who benefits? Who is harmed? These questions run through every chapter.

The loop that structures everything

Every chapter in this course follows the same arc:

retrieve → **parse** → **analyze**

- **Retrieve** a page or API response with **requests**
- **Parse** it into records — HTML, JSON, XML, or PDF
- Load into a **pandas** DataFrame
- **Analyze** or visualize the result

The specifics change; the arc does not. Recognizing it early gives you a mental model for every new chapter.



A first pipeline: CU Boulder's daily Wikipedia pageviews.

The data science mindset

Technical fluency is not enough. Four dispositions will serve you all semester:

- **Growth mindset** — ability develops through effort. Treat each error as a learning signal, not evidence of inadequacy.
- **Computational thinking** — decompose, recognize patterns, abstract, and specify unambiguous steps (Wing 2006).
- **The hacker ethic** — sharing, openness, curiosity, and a bias toward action. The libraries you'll use exist because people shared their work.
- **Scientific norms** — communalism, skepticism, and reproducibility. Document your methods; question your data; report limitations.

This is normal

You will meet libraries you've never used and pages that resist your parsing. Frustration is the job, not a detour from it.

2. How this class works

The weekly rhythm

Starting next week, every week runs on the same three-beat rhythm:

Monday | Concept + notebook intro

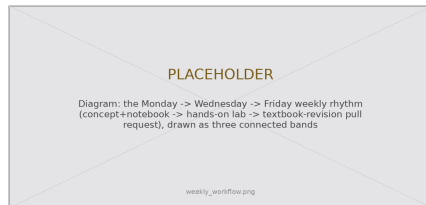
- Read the chapter; I introduce the week's ideas and the companion notebook

Wednesday | Notebook lab

- Hands-on — we work and debug the chapter's code together, live

Friday | Textbook revisions

- Propose and peer-review pull requests that improve the book



Concept, then practice, then contribution.

This week is the exception — Friday only. The full rhythm begins next week.

How you're evaluated

Your grade is **what you build**, not what you memorize:

- **Notebook Labs — 30%**
weekly, hands-on data-collection work
- **Textbook Revisions — 30%**
pull requests + peer reviews that improve the book
- **Final Project — 40%**
a research design + real data collection + analysis

No exams

No midterm, no final. You demonstrate learning by **doing the work of a web data scientist** — collecting data, contributing to a shared resource, and building a project.

Percentages are the course total; weekly labs are not individually high-stakes.

Documentation is the job

Your previous classes may have discouraged online resources. **The training wheels are off.** Finding, reading, and applying documentation is *the* core professional skill — not a shortcut, not cheating.

Bookmark the docs you'll live in: `requests`, `BeautifulSoup`, `pandas`, `matplotlib`.

“It’s not working” is not a question. Say what you **tried**, what you **expected**, what **happened**, and what the **error message** says.

Credit your sources

Used a blog post, a Q&A answer, docs, or an AI tool to solve something beyond class?

Just include a link in your code. Undocumented advanced tricks are a reliable signal of uncredited help.

3. Getting set up

Three things you'll need

Before Monday, get all three working on your own laptop:

1. **Python, via Anaconda** — the language plus a curated scientific stack and the conda package manager.
2. **Git** — version control that tracks every change and lets many people edit the same project safely.
3. **A free GitHub account** — where our textbook lives and where you'll submit revisions as pull requests.

Can't access a laptop or install software?

Contact me **immediately** so we can arrange an accommodation — do not wait until an assignment is due.

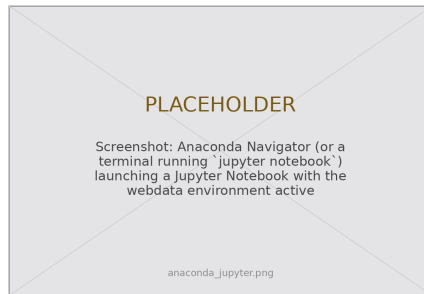
Anaconda, environments & core libraries

Install Anaconda from anaconda.com, then give this course its own isolated environment:

```
conda create -n webdata python=3.11
conda activate webdata
pip install requests beautifulsoup4 \
    pandas matplotlib seaborn
```

One environment per project records **exactly** which versions your analysis ran against — reproducibility, not just tidiness.

Launch your workspace with `jupyter notebook`: code, notes, and results in one file.



Anaconda launches Jupyter in your browser.

Your first request

Retrieving a web page programmatically is as simple as one call. Confirm your setup works:

```
import requests

url = "https://en.wikipedia.org/wiki/University_of_Colorado_Boulder"
response = requests.get(url)

print(response.status_code) # 200 means success
print(len(response.text)) # characters of raw HTML
```

Raw HTML isn't data yet. An API hands back JSON that drops straight into a DataFrame:

```
import pandas as pd
api = "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
url = f"{api}/en.wikipedia/all-access/all-agents/{article}/daily/20240101/20240131"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}

data = requests.get(url, headers=headers).json() # JSON -> dict
df = pd.DataFrame(data["items"]) # list of dicts -> DataFrame
```

Textbook Ch. 1, "Your First Request." The `User-Agent` header identifies you — a politeness norm we revisit next week.

Git & a free GitHub account

Git records the history of a project — who changed what, when, and why — and lets many people work on it without overwriting each other.

GitHub hosts Git projects online. Create a free account at `github.com`. Our textbook lives there as an open repository:

`github.com/cuinfoscience/Web-Data-Science-Book`

You'll **fork** it (make your own copy), edit a chapter, and propose your change back — exactly how open-source software is built.



The book is a living, open repository.

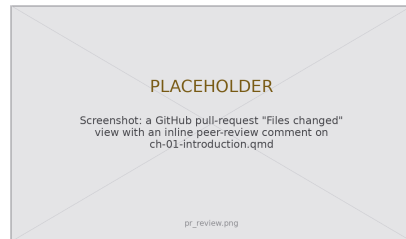
Sign up with your `colorado.edu` email to unlock free student features.

Textbook revisions as pull requests

Every Friday you improve the book itself. The workflow you'll repeat all semester:

1. **Fork / branch** the book repository
2. **Edit** a chapter's `.qmd` source; commit with a clear message
3. **Open a pull request** describing *what* you changed and *why*
4. **Review** two classmates' PRs — kind, specific, and actionable

A pull request is a proposal: your change sits side-by-side with the original so peers and I can comment line-by-line before it merges.



Contributions & reviews count toward the Textbook Revisions grade (30%). We walk through a real PR in Week 2.

The web is not a database.

It's a chaotic, evolving ecosystem of documents, APIs, and norms.

Your first challenge isn't modeling —
it's getting the data at all.

Key takeaways

1. Web data science is **retrieve** → **parse** → **analyze** — and the hard part is getting the data at all.
2. Every week has a rhythm: **Monday** concept, **Wednesday** notebook lab, **Friday** textbook-revision pull requests.
3. Your grade is what you build: Labs 30%, Revisions 30%, Final Project 40% — **no exams**.
4. Before Monday: install **Anaconda**, set up **Git**, and create a free **GitHub** account.
5. Documentation is a professional skill, not a shortcut — it's how you'll learn every new library.

Next week: **Ethics, law & responsible collection** — `robots.txt`, Terms of Service, and how to collect data without doing harm.

Reading: Textbook Ch. 1. Related: Salganik (2018); Wilson et al. (2017).